Phyto Nova — Complete Project Roadmap From Zero to Final Year Project

This roadmap explains how your Phyto Nova: Plant Disease Detection and AI Plant Care Assistant project was built, what each part does, what you have completed so far, how the parts connect, and what remains before you can confidently call the project complete.

I'll explain it from a beginner's perspective, including the programming concepts, commands, files, testing, machine-learning evaluation, Android APK installation, and final project report.

1. Understand the project first

Phyto Nova is a mobile application designed to help users identify possible plant diseases from leaf images and ask questions about plant care.

Your project has three major parts:

1. Mobile frontend

React Native + Expo + TypeScript

The screens users interact with: home/scanner, disease results, AI chatbot, settings, icons, and navigation.

2. Backend/API

Python + FastAPI + Uvicorn

Receives requests from the app, processes disease-prediction requests, and provides the chatbot endpoint.

3. Machine-learning model and AI service

The disease model predicts a class from a leaf image. The chatbot uses Google's Gemini API to answer plant-care questions.

These parts have different jobs. The disease classifier and the chatbot are not the same AI system: the classifier predicts a disease label, while the chatbot generates text.

2. Your project status so far

The following is based on the commands, logs, and code context you've shared. Anything not verified is marked as pending rather than assumed complete.

Working or configured

- React Native/Expo mobile project.
- Scanner connected to the backend's /predict endpoint.

- Chatbot screen and /chat endpoint; you previously reported an HTTP 200 response.
- Phyto Nova branding, settings screen, and bottom-tab icons.
- Android Expo configuration corrected.
- react-native-svg installed.
- EAS Build configured for Android.
- eas.json preview profile configured to build an APK.

Still to verify or complete

- Actual model accuracy, precision, recall, and F1-score.
- A reliable, labelled, independent test dataset.
- Android cloud build completed successfully and APK installed.
- Backend reachability from the installed APK.
- Testing scanner and chatbot together on a physical phone.
- Final documentation, screenshots, and project demonstration.

A successful API response or a successful app launch does not, by itself, prove that the disease model is accurate.

3. Phase 0 — Understand the basic concepts

Before following the technical steps, understand these terms.

Term	Simple meaning in Phyto Nova
Python	Language used for the backend and potentially model training.
JavaScript/TypeScript	Languages used to build the mobile app.
React Native	Lets you build mobile screens using React components.
Expo	Simplifies development and building of the React Native app.
API	A way for the frontend and backend to communicate.
FastAPI	Python framework that provides endpoints such as /predict and /chat.
Uvicorn	Runs the FastAPI server.
Machine learning	The model learns patterns from labelled examples.
Dataset	Collection of images and their correct labels.
Inference	Using a trained model to make a prediction.
Evaluation	Comparing predictions with known correct labels.
APK	An Android installation package.
EAS Build	Expo's cloud service for building mobile applications.

How the parts communicate

The frontend displays the response. The backend processes requests. The model classifies images. Gemini generates chatbot replies.

4. Phase 1 — Set up the development environment

This is the foundation you need before coding.

Step 1. Install the tools

Your Windows development environment uses:

- Python, for the backend.
- Node.js and npm, for JavaScript dependencies.
- Visual Studio Code, to edit project files.
- Expo, to run and develop the mobile app.
- Git, optionally, to track code changes.
- An Android phone with Expo Go for early testing, or an APK for standalone installation.

You have already reached the later stages of this setup.

Step 2. Create the project folders

Your project structure includes these locations:

```
PlantDiseaseProject/
├── api.py
├── .env
├── .venv/
└── PlantDiseaseApp/
    ├── app.json
    ├── eas.json
    ├── package.json
    └── src/
        ├── app/
        │   ├── index.tsx
        │   ├── chatbot.tsx
        │   ├── settings.tsx
        │   └── theme-context.tsx
        └── components/
            └── app-tabs.tsx
```

This is a simplified view of the known files, not necessarily every file in your project.

Why keep the backend and frontend separate?

- api.py handles requests and server-side logic.
- PlantDiseaseApp contains the mobile application.
- .venv holds Python packages for this project.
- .env stores configuration and secrets, such as the Gemini API key.
- app.json contains Expo app settings.

- eas.json controls cloud build profiles.

Keep .env and API keys private. Never upload secrets to a public repository.

Step 3. Create the Python environment

A virtual environment keeps the project's Python packages separate from other projects.

The command you use to run Python is:

```
cd C:\Users\hp\PlantDiseaseProject
.\.venv\Scripts\python.exe
```

To install packages, use the environment's Python and pip, for example:

```
.\.venv\Scripts\python.exe -m pip list
```

This lists installed packages. Do not reinstall everything without a reason if your current backend is working.

Step 4. Set up the mobile dependencies

The frontend has its own dependencies, managed by npm. You have used commands such as:

```
cd C:\Users\hp\PlantDiseaseProject\PlantDiseaseApp
npm.cmd install
```

Expo-compatible native dependencies should generally be installed using:

```
npx.cmd expo install package-name
```

You already used this approach for react-native-svg and expo-build-properties.

5. Phase 2 — Build or obtain the plant disease model

This is the most important machine-learning part of the project.

A mobile app cannot learn to recognise diseases simply because it has a scanner screen. It needs a trained model, a model file, or a service that performs the classification.

There are two possible situations for your project:

- Case A: You downloaded an already-trained plant disease model and integrated it into the backend.
- Case B: You trained your own model using a plant disease dataset.

We have not yet verified which applies to your current model, so do not claim that you trained a model yourself unless you can show the training code or records.

Step 1. Collect labelled leaf images

Each image must have a known correct class.

For example:

```
dataset/
├── train/
│   ├── Tomato___healthy/
│   ├── Tomato___Early_blight/
│   └── Tomato___Late_blight/
├── valid/
│   ├── Tomato___healthy/
│   ├── Tomato___Early_blight/
│   └── Tomato___Late_blight/
└── test/
    ├── Tomato___healthy/
    ├── Tomato___Early_blight/
    └── Tomato___Late_blight/
```

This is an example layout, not confirmation that these exact classes or folders exist in your project.

A labelled dataset tells the model which category each image belongs to. The model learns visual patterns associated with those categories.

Step 2. Split the dataset

A typical approach is to divide the data into:

- Training set: used to fit the model.
- Validation set: used to select settings and monitor training.
- Test set: used for the final independent evaluation.

An example split is 70% training, 15% validation, and 15% testing. These percentages are guidelines, not mandatory rules.

Avoid placing near-duplicate images or images of the same leaf in different splits, as that can make evaluation misleading.

Step 3. Preprocess the images

The input images must be processed in the same way during training and prediction. Typical operations include:

1. Resize the image to the model's required dimensions.
2. Convert the image to the expected colour format.
3. Normalize pixel values if required.
4. Convert the image into the model's expected tensor shape.

The exact input size, normalization, and class order depend on the model you are using. They must be taken from the model's actual configuration.

Step 4. Train the classifier, if needed

A common approach is transfer learning with a pretrained image classification model such as MobileNet, EfficientNet, or ResNet.

Transfer learning reuses visual features learned from a large image dataset and adapts the model to plant disease classes.

A typical training workflow is:

1 Load the labelled training and validation datasets.
2 Load a suitable pretrained model.
3 Replace the final classification layer with one matching your disease classes.
4 Train the new classification layer.
5 Fine-tune additional layers if appropriate.
6 Monitor validation loss and metrics.
7 Save the best model.
8 Evaluate it on the untouched test set.

This is a general workflow. We have not established that your current project uses any particular architecture.

Step 5. Save the model and class labels

The backend needs to know both the trained weights and the meaning of each output index.

For example, if a model outputs probabilities:

```
Index 0 → Tomato___Early_blight
Index 1 → Tomato___Late_blight
Index 2 → Tomato___healthy
```

That mapping must match the exact training configuration. If the class order is wrong, the app could display an incorrect disease name even when the model's numeric output is correct.

6. Phase 3 — Build the backend using FastAPI

Your backend file is:

```
C:\Users\hp\PlantDiseaseProject\api.py
```

The backend acts as the bridge between the mobile app, the disease model, and the chatbot service.

Step 1. Start the server

Your existing launch command is:

```
cd C:\Users\hp\PlantDiseaseProject
.\.venv\Scripts\python.exe -m uvicorn api:app --host 0.0.0.0 --port 8000
```

Here is what the command means:

- uvicorn starts the web server.
- api:app refers to the app object inside api.py.
- --host 0.0.0.0 allows the server to listen on available network interfaces.
- --port 8000 uses port 8000.

The server must remain running while your app sends requests.

Step 2. Understand the /predict endpoint

Your scanner calls the backend's /predict endpoint.

The intended flow is:

1. The user takes or selects a leaf image.
2. The frontend sends the image to the backend.
3. The backend reads the image.
4. The image is preprocessed for the classifier.
5. The model predicts a class.
6. The backend converts that output into a meaningful result.
7. The frontend displays the result.

The exact request format and response fields must be confirmed from api.py and index.tsx.

Step 3. Understand the /chat endpoint

Your chatbot uses /chat. You previously reported that it returned HTTP 200, meaning that request was handled successfully.

The intended flow is:

1. The user enters a plant-care question.
2. The frontend sends a JSON request to /chat.
3. The backend processes the message.
4. The backend sends the relevant prompt to Gemini.
5. Gemini generates a reply.
6. The backend returns the reply.
7. The frontend displays it in the chat.

The chatbot depends on network access and a valid Gemini API configuration. A working chatbot does not automatically establish that the image classifier works correctly.

Step 4. Test the backend independently

On your computer, open:

```
http://127.0.0.1:8000/docs
```

If FastAPI's interactive documentation is available, use it to inspect the endpoints and their request formats.

Test /predict using an appropriate leaf image, and test /chat using a simple plant-care question. Check the response and server logs for errors.

Do not change the scanner or /predict endpoint unnecessarily while it is working.

7. Phase 4 — Build the mobile frontend

Your mobile frontend lives in:

```
C:\Users\hp\PlantDiseaseProject\PlantDiseaseApp
```

It is built with React Native and Expo.

Step 1. Home screen and scanner

File: src/app/index.tsx

The home/scanner screen is responsible for the user interaction around image selection or capture and displaying the disease prediction.

The UI should handle:

- Camera or gallery permission, where applicable.
- Selecting or capturing an image.
- Showing a loading indicator during prediction.
- Displaying the returned class and any confidence score the backend provides.
- Handling server errors without crashing.
- Letting the user try another image.

The frontend's job is not to calculate the disease model's accuracy. It displays an individual prediction.

Step 2. Chatbot screen

File: src/app/chatbot.tsx

The chatbot screen includes:

- A message list.
- An input field.
- A send button.
- Quick question suggestions.

- A loading state while waiting for the response.
- A connection-error message if the server cannot be reached.

The chat screen currently uses the API base URL:

```
const API_URL = 'http://192.168.137.1:8000';
```

This is a local-network address. It is not a public production server address.

Step 3. Settings screen

File: src/app/settings.tsx

Your settings screen includes preferences and app information. We have seen the settings code, but a functional dark-mode switch should be tested to confirm it actually updates the app's appearance consistently.

A switch that changes its own value without changing the rest of the app is only a UI control, not a complete theme feature.

Step 4. Shared theme context

File: src/app/theme-context.tsx

A theme context can provide shared theme state to multiple screens. If you want persistent dark mode, the selected setting must also be saved and restored after restarting the app.

Step 5. Bottom-tab navigation

File: src/components/app-tabs.tsx

Your tabs use icons for the main screens. The navigation should be tested to verify that:

- Each icon opens the correct screen.
- The active tab is clearly indicated.
- The layout remains usable on different screen sizes.
- Theme colours remain consistent across tabs.

Step 6. App branding

Your Expo configuration is in app.json. It includes the app name, slug, icon, Android package identifier, splash screen configuration, and plugins.

Your displayed brand is Phyto Nova. If you want that name to appear beneath the installed Android icon, the Expo name field should be set to the desired display name, then the APK should be rebuilt.

8. Phase 5 — Connect the frontend and backend

This is where the mobile app becomes a complete system rather than a collection of screens.

Step 1. Configure the API URL

The frontend must point to an address that the phone can reach.

When the backend runs on your Windows computer, 127.0.0.1 inside the phone refers to the phone itself, not your computer. Therefore, a computer's reachable LAN address or a deployed server address is needed.

Step 2. Keep the backend running

Open one PowerShell window for the backend:

```
cd C:\Users\hp\PlantDiseaseProject
.\.venv\Scripts\python.exe -m uvicorn api:app --host 0.0.0.0 --port 8000
```

Keep it open.

Step 3. Start Expo in a second window

```
cd C:\Users\hp\PlantDiseaseProject\PlantDiseaseApp
npx.cmd expo start
```

For development, open the app through Expo Go or the appropriate development build.

Step 4. Test both features

Run at least these tests:

Test	Expected outcome
Open the app	Home screen loads.
Select a leaf image	Image selection works.
Predict a disease	Backend returns a result.
Select a healthy leaf	Model returns its healthy class if supported.
Send a chatbot question	A reply is displayed.
Stop the backend	App shows a useful connection error.
Restart the backend	Requests work again.

Test on the actual phone, not only in the computer browser. A backend working on the computer can still be unreachable from the phone.

9. Phase 6 — Evaluate the model scientifically

This phase is essential if you want to present Phyto Nova as a machine-learning project, especially in a college report, viva, or demonstration.

Step 1. Identify the model being evaluated

First establish:

- Which model file is loaded by api.py.
- Which disease classes it predicts.
- How images are preprocessed.
- How the output index is converted to a disease label.
- Whether the model is custom-trained or pretrained.

Do not assume that the chatbot's performance says anything about the disease classifier's performance.

Step 2. Prepare the test dataset

Use a labelled dataset with images that were not used to train or select the model.

For every test image, record:

- The image filename.
- The actual disease class.
- The predicted disease class.
- Optionally, the model's confidence for its predicted class.

For example:

Image	Actual class	Predicted class
leaf_001.jpg	Tomato early blight	Tomato early blight
leaf_002.jpg	Tomato healthy	Tomato healthy
leaf_003.jpg	Tomato late blight	Tomato early blight

These are illustrative examples, not results from your actual model.

Step 3. Understand the confusion matrix

For each disease class, the model can make four kinds of outcomes when that class is treated as the positive class:

- True positive (TP): correctly predicts that class.
- False positive (FP): predicts that class when the true class is different.
- False negative (FN): fails to identify an image that belongs to that class.
- True negative (TN): correctly identifies an image as not belonging to that class.

Step 4. Know the formulas

Accuracy

```
Correct predictions / All predictions
```

Measures overall correctness.

Precision

```
TP / (TP + FP)
```

Measures how trustworthy positive predictions are for a particular class.

Recall

```
TP / (TP + FN)
```

Measures how many actual examples of that class are detected.

F1-score

```
2 × (Precision × Recall) / (Precision + Recall)
```

Balances precision and recall.

When there are multiple disease classes, calculate the metrics per class and also report an average. Macro averages give each class equal weight. Weighted averages account for the number of test examples in each class. For a project report, providing both per-class results and macro averages is often informative.

Step 5. Run an evaluation script

Once the model file, preprocessing, class labels, and test dataset are confirmed, create an evaluation script, for example evaluate_model.py.

The script should:

1 Load the exact model used by the scanner.

2 Load the labelled test images.

3 Apply the correct preprocessing.

4 Run inference on each image.

5 Convert the outputs into class predictions.

6 Compare predicted labels with true labels.

7 Calculate accuracy, precision, recall, and F1-score.

8 Generate a classification report and confusion matrix.

9 Save the results for your project documentation.

The metric code shared earlier is the calculation portion; the model-loading and dataset-reading portion must be tailored to your actual files.

Important: Never report invented scores. If the model predicts 87% accuracy on a properly held-out test set, report that measured result—not a target value or a score observed during training.

Step 6. Evaluate real-world limitations

A model tested on clean, centred leaves may perform worse on phone photos because of:

- Different lighting.
- Blurry images.
- Multiple leaves in one image.
- Complex backgrounds.
- Unseen crop varieties.
- Diseases not included in the training classes.
- Similar-looking symptoms caused by different problems.

A confidence score is not automatically a probability that the diagnosis is correct. The model also needs testing on realistic phone images before its reliability can be claimed.

10. Phase 7 — Build the Android APK

You have already completed several setup steps for this phase.

Step 1. Correct the Expo configuration

You fixed the invalid android.usesCleartextTraffic field and configured the HTTP setting using the expo-build-properties plugin.

Your Expo Doctor output reached 19/20 checks. The remaining warning was that react-native-fast-tflite is listed as untested on React Native's New Architecture. That warning does not prove the package is incompatible, but an actual build is still needed to confirm.

Step 2. Configure EAS Build

You ran:

```
npx.cmd eas-cli build:configure
```

and Expo reported that the project was ready to build.

Step 3. Configure an APK profile

Your eas.json already contains a preview profile:

```
{
  "build": {
    "preview": {
      "distribution": "internal",
      "android": {
        "buildType": "apk"
      }
    },
    "production": {
      "android": {
        "buildType": "app-bundle"
      }
    }
  }
}
```

The preview profile is configured for an APK, which can be installed directly on an Android phone.

Step 4. Start the cloud build

After logging in to your Expo account, run:

```
npx.cmd eas-cli build --platform android --profile preview
```

Follow any prompts. EAS uploads the project and builds it in the cloud. Wait for the build result and open the provided build page.

Step 5. Install the APK

When the build succeeds:

1 Open the build page on your Android phone.

2 Download the APK.

3 Open the downloaded file.

4 If Android asks, allow installation from that source.

5 Tap Install.

6 Open Phyto Nova and test the scanner and chatbot.

If the build fails, save the complete error output. Fix the actual error rather than changing unrelated dependencies.

Step 6. Understand the local backend limitation

Your current backend address is http://192.168.137.1:8000. This is a private network address, not a public server.

That means your APK may only be able to use the scanner and chatbot when:

- The backend is running on your computer.
- The phone can reach the computer over the network.
- The address and port are correct.

- Windows Firewall permits the connection.

If you want Phyto Nova to work when your computer is off or your phone is on mobile data, you will need to deploy the backend to a reachable server and configure the app to use that server. For production, use HTTPS and secure API credentials.

11. Phase 8 — Quality assurance and final testing

Before calling the project complete, test the whole application—not just whether it opens.

Final testing checklist

Mark items as you verify them.

0/14

☐	App opens and navigation works
☐	Camera or gallery permissions work correctly
☐	Valid leaf images produce predictions
☐	Healthy leaves are handled correctly
☐	Unsupported or poor-quality images show useful errors
☐	Disease labels match the model's class mapping
☐	Chatbot returns useful responses
☐	Network failures show understandable messages
☐	The app remains usable while requests are loading
☐	The installed APK works without Expo Go
☐	The app is tested on a real Android phone
☐	Metrics are calculated using an independent labelled test set
☐	API keys are not embedded in public source code
☐	Final screenshots and test records are saved

This checklist is a planning aid, not a claim that these tests have already passed.

12. Phase 9 — Prepare the final project report

A good project report explains what you built, how you built it, and what evidence supports your results.

Recommended report structure:

1. Abstract

Summarise the problem, proposed app, technology, and measured results.

2. Introduction

Explain plant diseases, the need for early identification, and the project's motivation.

3. Problem statement and objectives

State what Phyto Nova aims to solve and what is within its scope.

4. Literature review

Discuss relevant plant disease classification approaches and related applications.

5. Requirements and system design

Describe the mobile app, backend, model, APIs, and architecture.

6. Dataset and methodology

Document the dataset source, classes, splits, preprocessing, model, and training method.

7. Implementation

Explain the React Native frontend, FastAPI backend, disease prediction, chatbot, and APK build.

8. Testing and evaluation

Include test cases, confusion matrix, accuracy, precision, recall, and F1-score.

9. Results and discussion

Show screenshots, measured results, errors, strengths, and limitations.

10. Conclusion and future work

Summarise achievements and identify realistic improvements.

11. References and appendix

List sources, package documentation, test records, and relevant code excerpts.

What screenshots should you include?

Capture real screenshots of:

- The Phyto Nova home/scanner screen.
- An example prediction.
- The chatbot responding to a question.
- The settings screen.
- The API documentation, if useful.
- The installed app running on the phone.
- Evaluation outputs, including the classification report and confusion matrix.

Do not include a screenshot of a made-up accuracy result. The final results section should contain your actual measured values.

13. Phase 10 — Final delivery and demonstration

At the end of the project, organise your deliverables:

Deliverable	Purpose
Android APK	Installable mobile application.
Source code	Frontend, backend, and evaluation script.
Model file and class mapping	Required model assets, subject to licensing and size constraints.
Dataset details	Source, labels, split strategy, and permissions.
Evaluation results	Reproducible metrics and confusion matrix.
Project report	Written explanation of the system and results.
Demonstration slides	Concise presentation for your review or viva.

Keep secrets out of the submitted source code. If the model or dataset comes from a third party, follow its licence and attribution requirements.

For the final demonstration, a good sequence is:

1. Explain the problem and project objectives.
2. Show the system architecture.
3. Open the installed Android app.
4. Scan a leaf image and show the result.
5. Ask a plant-care question in the chatbot.
6. Explain the model evaluation method and actual scores.
7. Discuss limitations and future improvements.

14. Recommended future improvements

Once the core application is reliable, consider these improvements in order of value:

1. Independent model evaluation: establish trustworthy metrics before making claims about accuracy.
2. Better real-world testing: test phone images captured under varied conditions.
3. Model confidence and uncertainty: communicate uncertain predictions clearly and avoid pretending every image can be classified reliably.
4. Improved error handling: handle invalid images, unavailable servers, and API failures.
5. Public backend deployment: allow the app to work without relying on your computer's local network.
6. Security: use HTTPS, protect secrets, and add suitable request limits and validation.
7. User experience: improve accessibility, loading indicators, and result explanations.

8 Disease-specific guidance: provide carefully sourced treatment recommendations without overclaiming diagnosis.

9 Persistence and history: optionally let users save past scans and compare changes over time.

10 Monitoring and maintenance: track failures, update dependencies carefully, and retest after model or backend changes.

These are future improvements, not requirements to complete all at once.

15. Your exact next steps from where you are now

You don't need to start the project again from zero. Continue from your current codebase.

Step 1 — Identify the model

Find the model file, the model-loading code in api.py, and the class-label mapping. Establish whether the model is pretrained or custom-trained.

Step 2 — Evaluate the classifier

Locate the test dataset or create a valid independent test split. Build and run evaluate_model.py using your actual model.

Step 3 — Finish the Android build

Run the configured EAS preview build, resolve any build errors, download the APK, and install it on your phone.

Step 4 — Verify end-to-end operation

Test both scanning and chatbot responses using the installed app, with the backend running and reachable.

Step 5 — Document the evidence

Save real metrics, test cases, screenshots, limitations, and build information for your report and demonstration.

My recommendation: start with Step 1. In PowerShell, run:

```
cd C:\Users\hp\PlantDiseaseProject
Get-ChildItem -Recurse -File -Include *.keras,*.h5,*.tflite,*.pth,*.pt,*.onnx
```

Then inspect the backend file:

```
Get-Content api.py
```

Remove any secrets before sharing the output. With those details, we can identify the actual model and build the evaluation workflow around your existing project instead of guessing at its architecture.

The key distinction is that your mobile app can be functional before its machine-learning performance has been scientifically measured. Completing the evaluation and validating the installed APK are the two major steps that will turn your current working prototype into a better-documented, tested project.